Workflows

Vance Russell[11] 3point.xyz

Abstract

Outlines the foundational principles for creating compelling and accurate data visualizations. These principles are grounded in cartographic theory, information design, and empirical research in visual perception.

This chapter outlines practical, reusable workflows for common geospatial visualization tasks using Python, GDAL, QGIS, and Leafmap.

1. Workflow 1: Loading and Exploring Geospatial Data

Goal: Quickly understand the structure, extent, and content of your data before visualization.

1.a. Common Formats

- **Vector:** Shapefiles (.shp), GeoJSON (.geojson), GeoPackage (.gpkg), PostGIS databases
- **Raster:** GeoTIFF (.tif), HDF5 (.h5), NetCDF (.nc), COG (Cloud Optimized GeoTIFF)

1.b. Tools & Approach

Option A: Python/GeoPandas (Best for exploration)

```
import geopandas as gpd
import rasterio
import pandas as pd

# Load vector data
gdf = gpd.read_file('data.shp')
print(gdf.head())
print(gdf.crs)
print(gdf.bounds)
print(gdf.info())

# Load raster data
with rasterio.open('data.tif') as src:
    print(f"Shape: {src.shape}")
    print(f"CRS: {src.crs}")
    print(f"Bounds: {src.bounds}")
    print(f"Bands: {src.count}")
```

Published Apr 16, 2026

Option B: GDAL Command Line (Best for quick inspection)

```
# Vector inspection
ogrinfo data.shp
ogrinfo data.shp layer_name

# Raster inspection
gdalinfo data.tif
```

Option C: QGIS (Best for visual exploration)

- Drag and drop files into QGIS
- Right-click layers arrow.r Properties to inspect attributes and styling options
- Use Identify tool to explore feature attributes

1.c. Key Questions to Answer

1. What CRS is my data in? Do I need to reproject?
2. What are the data bounds/extent?
3. What attributes/bands do I have?
4. What are the data types and ranges?
5. Are there any missing values or gaps?

---

2. Workflow 2: Data Preparation and Cleaning

Goal: Prepare messy raw data for visualization by standardizing projections, fixing geometries, and handling missing values.

2.a. Common Tasks

Reproject to a Common CRS

```
import geopandas as gpd

gdf = gpd.read_file('data.shp')
print(f"Original CRS: {gdf.crs}")

# Reproject to Web Mercator (standard for web maps)
gdf_reprojected = gdf.to_crs(epsg=3857)

# Or reproject to local UTM zone (for accurate measurements)
gdf_utm = gdf.to_crs(epsg=32633)  # UTM zone 33N
```

Fix Invalid Geometries

```
# Check for invalid geometries
print(gdf[~gdf.geometry.is_valid])

# Fix invalid geometries using buffer(0)
gdf['geometry'] = gdf.geometry.buffer(0)

# Alternative: use make_valid from shapely
from shapely.ops import unary_union
gdf['geometry'] = gdf.geometry.apply(lambda x: x if x.is_valid else
x.buffer(0))
```

Handle Missing Values

```
# Check for null geometries
print(gdf[gdf.geometry.is_null])

# Drop rows with null geometries
gdf = gdf[~gdf.geometry.is_null]

# Fill or remove rows with missing attribute data
gdf = gdf.dropna(subset=['important_column'])
```

Filter/Subset Data

```
# By bounding box
bbox = (min_lon, min_lat, max_lon, max_lat)
gdf_subset = gdf.cx[bbox[0]:bbox[2], bbox[1]:bbox[3]]

# By attribute value
gdf_subset = gdf[gdf['category'] == 'target_value']

# By spatial intersection
gdf_subset = gpd.sjoin(gdf, study_area, how='inner',
predicate='intersects')
```

GDAL Approach (Command Line)

```
# Reproject and simplify in one step
ogr2ogr -t_srs EPSG:3857 -simplify 10 output.shp input.shp

# Convert between formats
ogr2ogr -f "GeoJSON" output.geojson input.shp

# Filter features
ogr2ogr -where "attribute = 'value'" output.shp input.shp
```

---

3. Workflow 3: Vector Data Analysis & Spatial Operations

Goal: Derive new insights through spatial relationships, aggregations, and geometric operations.

3.a. Common Operations

Spatial Join (overlay analysis)

```
# Find which features from one layer intersect another
points_in_polygons = gpd.sjoin(points, polygons, how='inner',
predicate='within')

# Count features per polygon
points_count =
points_in_polygons.groupby('polygon_id').size().reset_index(name='count')
polygons_with_count = polygons.merge(points_count, on='polygon_id')
```

Buffer and Distance Analysis

```
# Create buffer zones
gdf['buffer_100m'] = gdf.geometry.buffer(100)  # 100 meter buffer

# Find features within distance
gdf_buffered = gpd.GeoDataFrame(geometry=gdf['buffer_100m'],
crs=gdf.crs)
nearby_features = gpd.sjoin(other_layer, gdf_buffered,
predicate='intersects')
```

Dissolve/Aggregate Geometries

```
# Dissolve by attribute (combine polygons)
gdf_dissolved = gdf.dissolve(by='category', aggfunc='sum')

# Dissolve all into single geometry
gdf_merged = gdf.dissolve()

# Dissolve with custom aggregation
gdf_agg = gdf.dissolve(by='county',
                       aggfunc={'population': 'sum',
                                'area': 'first'})
```

Clip and Intersect

```
# Clip features to study area
study_area = gpd.read_file('study_boundary.shp')
gdf_clipped = gpd.clip(gdf, study_area)

# Find intersection of two layers
intersection = gpd.overlay(gdf1, gdf2, how='intersection')
```

Create Choropleth Data (classification)

```
# Natural breaks (Jenks)
from jenkspy import jenks_breaks
breaks = jenks_breaks(gdf['value'], n_classes=5)
gdf['class'] = pd.cut(gdf['value'], bins=breaks)

# Quantiles (equal counts per class)
gdf['class'] = pd.qcut(gdf['value'], q=5)

# Manual breaks
breaks = [0, 100, 500, 1000, 5000]
gdf['class'] = pd.cut(gdf['value'], bins=breaks)
```

QGIS Approach

- Vector arrow.r Geoprocessing Tools arrow.r Buffer, Dissolve, Clip, etc.
- Vector arrow.r Analysis Tools arrow.r Spatial join, intersection
- Processing toolbox for batch operations

---

4. Workflow 4: Raster Data Processing & Analysis

Goal: Process, analyze, and extract insights from raster imagery and gridded data.

4.a. Common Tasks

Load and Inspect Raster Data

```
import rasterio
from rasterio.plot import show
import numpy as np

with rasterio.open('data.tif') as src:
    # Read specific band
    band1 = src.read(1)

    # Read all bands
    data = src.read()

    # Get metadata
    print(f"CRS: {src.crs}")
    print(f"Shape: {src.shape}")
    print(f"Transform: {src.transform}")

    # Display
    show(band1)
```

Raster Calculations (Band Math)

```
# NDVI calculation from sentinel bands
with rasterio.open('sentinel_B4.tif') as red, \
     rasterio.open('sentinel_B8.tif') as nir:

    red_band = red.read(1).astype(float)
    nir_band = nir.read(1).astype(float)

    # Calculate NDVI
    ndvi = (nir_band - red_band) / (nir_band + red_band)

    # Save result
    profile = red.profile
    profile.update(dtype=rasterio.float32)

    with rasterio.open('ndvi.tif', 'w', **profile) as dst:
        dst.write(ndvi.astype(rasterio.float32), 1)
```

Reproject Raster

```
from rasterio.warp import calculate_default_transform, reproject,
Resampling

with rasterio.open('data.tif') as src:
    # Calculate transform for target CRS
    transform, width, height = calculate_default_transform(
```

```
        src.crs, 'EPSG:3857', src.width, src.height, *src.bounds)

    # Create output dataset
    kwargs = src.meta.copy()
    kwargs.update({
        'crs': 'EPSG:3857',
        'transform': transform,
        'width': width,
        'height': height
    })

    with rasterio.open('reprojected.tif', 'w', **kwargs) as dst:
        for i in range(1, src.count + 1):
            reproject(
                rasterio.band(src, i),
                rasterio.band(dst, i),
                src_transform=src.transform,
                src_crs=src.crs,
                dst_transform=transform,
                dst_crs='EPSG:3857',
                resampling=Resampling.nearest)
```

Clip Raster to Extent

```
import rasterio.mask

with rasterio.open('data.tif') as src:
    # Clip to bounding box
    bbox = [(min_lon, min_lat), (max_lon, min_lat),
            (max_lon, max_lat), (min_lon, max_lat), (min_lon, min_lat)]

    clipped, transform = rasterio.mask.mask(src, [bbox], crop=True)

    # Save clipped raster
    profile = src.profile
    profile.update({'height': clipped.shape[1],
                    'width': clipped.shape[2],
                    'transform': transform})

    with rasterio.open('clipped.tif', 'w', **profile) as dst:
        dst.write(clipped)
```

GDAL Command Line Approach

```
# Reproject raster
gdalwarp -t_srs EPSG:3857 input.tif output.tif

# Clip to extent
gdal_translate -projwin 0 10000000 10000000 0 input.tif output.tif

# Calculate NDVI-like operations
gdal_calc.py -A B4.tif -B B8.tif --outfile=NDVI.tif --calc="(B-A)/(B+A)"
```

5. Workflow 5: Static Map Visualization

Goal: Create publication-quality maps for print or static web display.

5.a. Python/Geopandas Approach

Basic Choropleth Map

```
import geopandas as gpd
import matplotlib.pyplot as plt
import contextily as ctx

# Load data
gdf = gpd.read_file('data.shp')

# Create figure
fig, ax = plt.subplots(figsize=(12, 10))

# Plot choropleth
gdf.plot(column='value',
         ax=ax,
         legend=True,
         cmap='viridis',
         edgecolor='k',
         alpha=0.8)

# Add basemap
ctx.add_basemap(ax, crs=gdf.crs, zoom=10)

# Formatting
ax.set_xlabel('Longitude')
ax.set_ylabel('Latitude')
ax.set_title('My Map Title', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig('map.png', dpi=300, bbox_inches='tight')
plt.show()
```

Multi-Layer Map

```
fig, ax = plt.subplots(figsize=(14, 10))

# Background layer
background.plot(ax=ax, color='lightgray', alpha=0.3)

# Main data layer
gdf.plot(column='value', ax=ax, cmap='RdYlGn',
         edgecolor='black', linewidth=0.5,
         legend=True, alpha=0.9)

# Overlay points
points.plot(ax=ax, color='red', markersize=30, marker='*')
```

```
# Add labels for specific features
for idx, row in gdf.iterrows():
    ax.text(row.geometry.centroid.x, row.geometry.centroid.y,
            row['label'], fontsize=8, ha='center')

ctx.add_basemap(ax, crs=gdf.crs, source=ctx.providers.CartoDB.Positron,
zoom=12)
plt.title('Multi-layer Analysis Map')
plt.savefig('multimap.png', dpi=300)
```

Raster + Vector Overlay

```
import rasterio
from rasterio.plot import show

fig, ax = plt.subplots(figsize=(12, 10))

# Display raster
with rasterio.open('raster.tif') as src:
    show(src, ax=ax, cmap='terrain', alpha=0.7)

# Overlay vector
gdf.plot(ax=ax, facecolor='none', edgecolor='red', linewidth=2)

ax.set_title('Raster + Vector Visualization')
plt.savefig('raster_vector.png', dpi=300)
```

5.b. QGIS Approach (Production Cartography)

1. **Load all layers** and apply symbology
2. **Project arrow.r New Print Layout** (or Print Composer)
3. **Add map frame**, scale bar, north arrow, legend
4. **Export as PDF or high-res image**
5. For professional output, export to Adobe Illustrator (.eps) for final touches

5.c. Key Considerations

- **Resolution:** 300 DPI for print, 72-96 DPI for web
- **Color scheme:** Use colorblind-friendly palettes
- **Scale:** Include scale bar or map coordinates
- **Legend:** Clear, non-overlapping with data
- **Title & metadata:** Document data sources and date

---

6. Workflow 6: Interactive Web Map Visualization

Goal: Create interactive maps for web browsers and Jupyter notebooks.

6.a. Leafmap Approach (Recommended for this course)

Basic Interactive Map

```
import leafmap

# Create map centered on location
```

```
m = leafmap.Map(center=(40, -100), zoom=4, height='600px')

# Add basemaps
m.add_basemap('OpenStreetMap')

# Add vector layer
m.add_shp('data.shp', layer_name='My Shapefile', zoom_to_layer=True)

# Add raster layer
m.add_raster('data.tif', layer_name='My Raster')

# Display
m.show()
```

Styled Vector Layer with Pop-ups

```
import leafmap
import geopandas as gpd

m = leafmap.Map(center=(40, -100), zoom=4)

# Convert to GeoJSON for better styling
gdf = gpd.read_file('data.shp')
geojson = gdf.__geo_interface__

# Add with styling
style = {
    'fillColor': '#ff0000',
    'color': 'black',
    'weight': 1,
    'opacity': 0.7,
    'fillOpacity': 0.5
}

m.add_geojson(geojson, layer_name='Styled Data', style=style)

# Display
m.show()
```

Choropleth Map (color by attribute)

```
import leafmap
import geopandas as gpd

m = leafmap.Map(zoom=10)

gdf = gpd.read_file('data.shp')

# Convert to GeoJSON
geojson = gdf.__geo_interface__

# Add choropleth
m.add_geojson(
```

```
    geojson,
    layer_name='Choropleth',
    fill_colors=['blue', 'lime', 'red']  # Color ramp
)

m.show()
```

Multiple Layers with Toggling

```
m = leafmap.Map(center=(40, -100), zoom=4)

# Add multiple layers
m.add_basemap('OpenStreetMap')
m.add_shp('layer1.shp', layer_name='Layer 1')
m.add_shp('layer2.shp', layer_name='Layer 2')
m.add_raster('raster.tif', layer_name='Raster Data')

# Layers are automatically toggleable
m.show()
```

6.b. Folium Approach (Alternative)

```
import folium
import geopandas as gpd

# Create map
m = folium.Map(location=[40, -100], zoom_start=4)

# Add data
gdf = gpd.read_file('data.shp')

# Convert to GeoJSON and add
folium.GeoJson(gdf.__geo_interface__).add_to(m)

# Save to HTML
m.save('map.html')
```

6.c. Key Features to Add

- **Legends:** Explain color schemes and symbols
- **Popups:** Click features to see attributes
- **Tooltips:** Hover to see information
- **Layer controls:** Toggle layers on/off
- **Scale bar:** Add map scale
- **Search functionality:** Find locations or features

6.d. Export Options

```
# Save to HTML file
m.to_html('output_map.html')

# Save to static image (requires additional setup)
# m.to_image('output_map.png')
```

---

7. Workflow 7: Multi-Source Data Integration

Goal: Combine data from multiple sources for comprehensive analysis.

7.a. Common Scenarios

Spatial Join Different Datasets

```
import geopandas as gpd

# Load datasets
boundaries = gpd.read_file('boundaries.shp')
points = gpd.read_file('points.shp')
polygons = gpd.read_file('polygons.shp')

# Ensure same CRS
points = points.to_crs(boundaries.crs)
polygons = polygons.to_crs(boundaries.crs)

# Join: find which points are in which boundaries
points_with_boundary = gpd.sjoin(points, boundaries,
                                 how='left', predicate='within')

# Aggregate: count features per boundary
point_counts = points_with_boundary.groupby('boundary_id').size()
boundaries = boundaries.join(point_counts.rename('point_count'))

# Combine geometries: merge raster data with vector
area_stats = gpd.sjoin(boundaries, polygons, how='left')
```

Time Series Data

```
import geopandas as gpd
import pandas as pd
import matplotlib.pyplot as plt

# Load multiple time periods
data_2010 = gpd.read_file('data_2010.shp')
data_2015 = gpd.read_file('data_2015.shp')
data_2020 = gpd.read_file('data_2020.shp')

# Add year field
data_2010['year'] = 2010
data_2015['year'] = 2015
data_2020['year'] = 2020

# Combine
combined = pd.concat([data_2010, data_2015, data_2020],
ignore_index=True)

# Analyze change
change = combined.pivot_table(values='value', index='id',
columns='year')
change['change_2010_2020'] = change[2020] - change[2010]
```

```
# Visualize
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
data_2010.plot(column='value', ax=axes[0], title='2010')
data_2015.plot(column='value', ax=axes[1], title='2015')
data_2020.plot(column='value', ax=axes[2], title='2020')
plt.tight_layout()
plt.show()
```

Attribute Table Enrichment

```
# Join external data
gdf = gpd.read_file('spatial_data.shp')
external = pd.read_csv('external_data.csv')

# Merge on common ID
enriched = gdf.merge(external, on='feature_id', how='left')

# Now you can visualize based on enriched attributes
enriched.plot(column='new_attribute', cmap='viridis')
```

---

8. Workflow 8: Automation & Batch Processing

Goal: Process multiple files or repeat tasks efficiently using scripts.

8.a. Batch Process Multiple Files

Process All Shapefiles in Directory

```
import os
import glob
import geopandas as gpd
from pathlib import Path

# Find all shapefiles
shapefiles = glob.glob('data/*.shp')

output_dir = 'processed_data'
os.makedirs(output_dir, exist_ok=True)

for shapefile in shapefiles:
    print(f"Processing {shapefile}...")

    # Load and process
    gdf = gpd.read_file(shapefile)

    # Clean data
    gdf = gdf[~gdf.geometry.is_null]
    gdf = gdf.to_crs(epsg=3857)

    # Save
    output_name = Path(shapefile).stem + '_processed.shp'
    gdf.to_file(os.path.join(output_dir, output_name))
```

```
    print(f"  ✓ Saved to {output_name}")
```

Generate Multiple Maps Automatically

```
import geopandas as gpd
import matplotlib.pyplot as plt
from pathlib import Path

# Load data with multiple categories
gdf = gpd.read_file('data.shp')

# Create separate map for each category
for category in gdf['type'].unique():
    subset = gdf[gdf['type'] == category]

    fig, ax = plt.subplots(figsize=(10, 10))
    subset.plot(ax=ax, alpha=0.7, edgecolor='k')

    ax.set_title(f'Map: {category}')

    # Save with category name
    filename = f'map_{category}.png'
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    plt.close()

    print(f"Saved {filename}")
```

8.b. GDAL Batch Processing (Command Line)

Reproject All GeoTIFFs

```
#!/bin/bash
# Save as process_rasters.sh

for file in data/*.tif; do
    echo "Reprojecting $file..."
    output=$(basename "$file" .tif)_3857.tif
    gdalwarp -t_srs EPSG:3857 "$file" "processed/$output"
done
```

Convert Multiple Formats

```
#!/bin/bash
# Convert all shapefiles to GeoJSON

for file in data/*.shp; do
    output=$(basename "$file" .shp).geojson
    ogr2ogr -f "GeoJSON" "$output" "$file"
    echo "Converted $file → $output"
done
```

8.c. QGIS Processing with Python

```python
from qgis.core import QgsProject, QgsVectorLayer
import processing
from qgis import core as qgis_core

# Initialize QGIS
qgis_core.QgsApplication.setPrefixPath("/usr", True)
qgis_app = qgis_core.QgsApplication([], False)
qgis_app.initQgis()

# Load project
project = QgsProject.instance()
project.read('project.qgs')

# Run processing algorithm
processing.run("qgis:buffer", {
    'INPUT': 'input_layer',
    'DISTANCE': 100,
    'OUTPUT': 'buffered_output.shp'
})

# Save
project.write()
qgis_app.exitQgis()
```

8.d. Schedule Automated Tasks

Cron job (Linux/Mac)

```cron
# Run script every day at 2 AM
0 2 * * * /home/user/process_data.py
```

Task Scheduler (Windows)

- Create scheduled task to run `.bat` file or Python script at specified intervals

---

9. Workflow Selection Guide

Choose your approach based on your task:

Task	Best Tool	Why
Exploring data	Python/GeoPandas	Quick, interactive, flexible
Visual inspection	QGIS	Intuitive GUI, fast feedback
Data cleaning	Python/GDAL	Automated, reproducible, batch-capable
Complex analysis	Python/GeoPandas	Powerful, scriptable, combines with other tools
Production cartography	QGIS or Python/Matplotlib	Professional output control
Interactive web maps	Leafmap/Folium	User-friendly, web-ready

Task	Best Tool	Why
Large-scale processing	GDAL/Command-line	Efficient, scriptable, minimal memory
Sharing results	Interactive (Leafmap) or Static (PNG/PDF)	Depends on audience and use case

---

10. Common Troubleshooting & Tips

10.a. CRS/Projection Issues

Problem: Layers don't overlap or coordinates are wrong

Solution:

```
# Always check CRS first
print(gdf.crs)

# Reproject to match
gdf = gdf.to_crs(reference_layer.crs)

# For web maps, use EPSG:3857 (Web Mercator)
gdf_web = gdf.to_crs(epsg=3857)
```

10.b. Invalid Geometries

Problem: Operations fail with geometry errors

Solution:

```
# Check for invalid geometries
invalid = gdf[~gdf.geometry.is_valid]
print(f"Found {len(invalid)} invalid geometries")

# Fix them
gdf['geometry'] = gdf.geometry.buffer(0)

# Or drop them
gdf = gdf[gdf.geometry.is_valid]
```

10.c. Memory Issues with Large Datasets

Problem: Out of memory errors processing big files

Solution:

```
# Process in chunks
from shapely.geometry import box

# Define grid of chunks
bounds = gdf.total_bounds
chunk_size = 10000  # meters

for x in range(int(bounds[0]), int(bounds[2]), chunk_size):
```

```
    for y in range(int(bounds[1]), int(bounds[3]), chunk_size):
        chunk_bounds = box(x, y, x + chunk_size, y + chunk_size)
        chunk = gpd.clip(gdf, chunk_bounds)
        # Process chunk
        process_and_save(chunk)

# Or use spatial indexing
from rtree import index
idx = index.Index()
```

10.d. Performance Optimization

Tip 1: Simplify geometries for visualization

```
# Reduce complexity for faster rendering
gdf['geometry'] = gdf.geometry.simplify(tolerance=10)
```

Tip 2: Use spatial index for queries

```
# Faster than iterating all features
from rtree import index
idx = index.Index()

for i, geom in enumerate(gdf.geometry):
    idx.insert(i, geom.bounds)

# Query nearby features
hits = list(idx.intersection(query_geometry.bounds))
```

Tip 3: Cache intermediate results

```
# Don't recalculate; save processed data
gdf_processed.to_file('cache/processed.shp')

# Load cached version next time
gdf_processed = gpd.read_file('cache/processed.shp')
```

10.e. Visualization Issues

Problem: Map looks blank or wrong colors

Solution:

```
# Check data range
print(gdf['column'].describe())

# Use appropriate color scheme
gdf.plot(column='value',
         cmap='viridis',  # Use colorblind-friendly palette
         vmin=gdf['value'].min(),
         vmax=gdf['value'].max())

# For categorical data
gdf.plot(categorical=True, cmap='tab10')
```

10.f. Export/Format Issues

Problem: Can't open exported file

Solution:

```
# Always specify driver explicitly
gdf.to_file('output.geojson', driver='GeoJSON')
gdf.to_file('output.shp', driver='ESRI Shapefile')
gdf.to_file('output.gpkg', driver='GPKG')

# For images
plt.savefig('map.png', dpi=300, format='png')
```

---

11. Quick Reference Cheat Sheet

11.a. Essential Commands

Python/GeoPandas

```
# Load
gdf = gpd.read_file('file.shp')

# Inspect
gdf.info(), gdf.head(), gdf.crs, gdf.bounds

# Transform
gdf = gdf.to_crs(epsg=3857)

# Filter
subset = gdf[gdf['field'] > 100]

# Spatial
result = gpd.sjoin(gdf1, gdf2, predicate='intersects')

# Visualize
gdf.plot(column='value', cmap='viridis')
```

GDAL Command Line

```
# Inspect
gdalinfo file.tif
ogrinfo file.shp

# Transform
gdalwarp -t_srs EPSG:3857 input.tif output.tif
ogr2ogr -t_srs EPSG:3857 output.shp input.shp

# Convert
gdal_translate -of GTiff input.tif output.tif
ogr2ogr -f "GeoJSON" output.geojson input.shp
```

Leafmap

```python
import leafmap

m = leafmap.Map(center=(40, -100), zoom=4)
m.add_shp('data.shp', layer_name='Layer')
m.add_raster('data.tif', layer_name='Raster')
m.show()
```

---

12. Next Steps

After mastering these workflows:

- Combine multiple workflows for complex analysis
- Build reusable functions and libraries
- Create interactive web applications
- Automate routine tasks with scheduling
- Integrate with other GIS tools and services

Remember the 5 Pillars from the Visualization Primer:

1. Purpose & Audience
2. Data Integrity
3. Appropriate Visual Encoding
4. Simplicity & Clarity
5. Visual Hierarchy & Design Integration

Apply these principles as you progress through these workflows to create effective visualizations!